Homework 3

Total Points: 100 (Correctness and Style) + 3 EC (Checkpoint) + 5 EC (Question 5.2)

Due Dates (SD time):

- **Entire Assignment: Monday, April 21st, 11:59pm**
- **Checkpoint (read below): Saturday, April 19th, 11:59pm**

Starter Files

Download hw03.zip. Inside the archive, you will find starter files for the questions of this homework. You cannot import anything to solve the problems.

IMPORTANT: Coding and Docstring Style

This is a reminder that your code style will be graded. Here are a few useful links:

Style Guide Document

Style Guide on Course Website

Style Guide Examples

Submission

By the end of this homework, you should submit the homework via Gradescope. You may submit more than once before the deadline; only the final submission will be graded. Refer to Lab00 directions for submission.

Testing

At any point of the homework, use one of the following command to test your work:

```
>>> python3 -m doctest hw03.py
>>> py -m doctest hw03.py
>>> python -m doctest hw03.py
```

Checkpoint Submission

Due Date: Saturday, April 19th, 11:59 PM (SD time)

You can earn up to **3 points extra credit** by submitting the checkpoint by the due date above. In the checkpoint submission, you should complete **Question 1, 2, 3** and submit ***hw03.py*** file to gradescope.

Checkpoint submission is graded by some simple hidden tests (no tests against edge cases). Note that in your final submission, you should still submit these questions, and you may modify your implementation if you noticed any errors.

A Guide on Asserts

Starting from HW03, you will be asked to write **assert statements** whenever a question requires them. Assert statements are used to prevent any unexpected input that may corrupt your code, including arguments in invalid type, and arguments that do not fit the logic and the input constraints. Refer to the [Style Guide Example]() for examples. Here are a few important notes to guide you:

1. You should check **each** input argument, so go through them one by one;
2. The requirements of an argument will be provided in the question prompt. For example, if the write-up says: take in a length-5 list of positive integers called `lst`, you should check the following **in order**:
    a. `lst` is a list
    b. `lst` has length 5
    c. The elements of `lst` are all integers
    d. The elements of `lst` are all positive
3. Note that order is important because some checks depend on the correct check result of previous ones. For example, if the length check happens before the list type check, an error will occur when the input is an integer, because you cannot apply the `len()` function on an integer.
4. For type checks, the functions `type()` or `isinstance()` are helpful.
5. For assertion error doctests, please follow examples given in the doctest. In particular, you could simply copy-paste the expected output for all assertion errors.
6. When you need to check everything inside a list, for example, you will need to loop through all elements. Instead of using an explicit for loop, you must use list comprehension with the `any()/all()` function to achieve this in this class. For example,

Incorrect	**Correct**
`for idx in index:`	`assert all([`

`assert isinstance(idx, int)`	`isinstance(idx, int)`
	`for idx in index`
	`])`

7. The tutorial on `any()`, `all()` function can be found here.
8. Assert statements do not count towards the number of lines in the one-line list comprehension requirement.
9. In Python, boolean values are integers (source). You can assume that in cases where you need to assert for integer types, booleans will NOT be tested.

Example (assert + magic number declaration + list comprehension requirement):

```
def increment(nums):
    """
    Docstring (and doctests) omitted.
    """
    increment_amount = 10
    assert isinstance(nums, list)
    return [num + increment_amount for num in nums]
```

General Notes and Requirements

1. **DO NOT IMPORT ANY PACKAGES.**
2. Please add your own doctests (**at least three**) as the given doctests are not sufficient. **You will be graded on the doctests**.
3. When a question **(Question 1, 2, 3, 4, 5.1, 5.2, 6)** requires **list comprehension:** your implementation should have list comprehension(s) and possible assignment lines, including magic number declarations. **No explicit for/while loops** are allowed. **No helper functions (including lambdas)** are allowed. **No map/filter** are allowed (will be covered later). For example, the following is allowed:

```
lst_1 = [... for ... in ...]
lst_2 = [... for ... in ...]
return ...
```

4. When a question **(Question 1, 2, 3)** requires **assert statements**: follow the assert guide to write assert statements to prevent any unexpected inputs. You will be graded on this. If a question does not require assert statements, assume all inputs are valid.

Question 1:

We have discussed using assert statements to test the correctness of user inputs, as users could give unpredictable inputs that could potentially break the function. In this question, you need to implement the assert statements to check **all possible incorrect inputs**, and return some results as required.

Question 1.1:

Write a function **`operate_nums`**`()` that takes a list of integers and returns a new list. The function doubles the value of **odd** integers and triples the value of **even** integers.

Requirements: Assert statements, [List Comprehension]

```
def operate_nums(lst):
    """
    >>> operate_nums([1, 2, 3, 's'])
    Traceback (most recent call last):
    ...
    AssertionError
    >>> operate_nums([2, 3.1, -2, 0, 5])
    Traceback (most recent call last):
    ...
    AssertionError
    >>> operate_nums([2, 3, -2, 0, 5])
    [6, 6, -6, 0, 10]
    """
```

Question 1.2:

Write a function **`string_lengths`**`()` that takes a list of non-empty strings and a list of positive integers of the same length. *Ensure* that the lengths of the lists are checked and not *assumed*.

The function returns a list of boolean values, where `True` indicates that the length of the string is strictly greater than the corresponding integer in the second list, and `False` otherwise.

Requirements: Assert statements, List Comprehension

```
def string_lengths(text, nums):
    """
    >>> string_lengths(['a', 'b', 'c'], [1, 2])
    Traceback (most recent call last):
    ...
    AssertionError
    >>> string_lengths(['', 'abc'], [1, 2])
    Traceback (most recent call last):
    ...
    AssertionError
    >>> string_lengths(['a', 'b'], [-1, 5])
    Traceback (most recent call last):
    ...
    AssertionError
    >>> string_lengths(['abc', 'abcd', 'abcde'], [2, 5, 5])
    [True, False, False]
    """
```

Question 1.3:

Write a function **`process_dict()`** that takes a dictionary where the keys are tuples and the values are lists of strings. The function should return a list of integers, where each integer represents the sum of:

- The length of the key (the tuple),
- The total length of the strings in the value (the list of strings).

Requirements: Assert statements, List Comprehension

Example:

Input Dictionary: `{(1, 2): ['dsc', 'dsc20', 'dsc30'], (2,): ['b']}`
Expected Output: `[15, 2]`

- For the first entry, we have key length 2 and total string length of 3 + 5 + 5 = 13, so the first element in the returning list is 2 + 13 = 15;
- For the second entry, we have key length 1 and total string length 1, so in the output list we put 2.

```python
def process_dict(input_dict):
    """
    >>> process_dict({1: ['a', 'b', 'c'], (1, 2): ['a']})
    Traceback (most recent call last):
    ...
    AssertionError
    >>> process_dict({(1, 2): ['a', 0], (2, ): ['b']})
    Traceback (most recent call last):
    ...
    AssertionError
    >>> process_dict({(1, 2): ['dsc', 'dsc20', 'dsc30'], (2,): \
    ['b']})
    [15, 2]
    """
```

Question 2:

Implement a function `unusual_sort` which takes in two lists of the same size:

- **`indices`** - list of length n that contains integers 0 to n-1
- **`items`** - list of length n

The function will sort list **`items`** such that the i-th element of the "sorted" list is the **`indices`**[i]th element in **`items`**.

For example, if **`indices = [0, 2, 1]`** and **`items = ['a', 'b', 'c']`**, the order of the "sorted" list will be **`[items[indices[0]], items[indices[1]], items[indices[2]]]`** which is **`['a', 'c', 'b']`**. More examples can be found in the given doctests.

The function will return:

- the sorted list as a list of tuples **(element, original index, new index)** of length n.
- Return an empty list if n = 0.

See the table below for a few examples:

Examples:

Input	Output	Reason

indices = [0, 4, 2, 3, 1] **items** = ["A", "E", "C", "D", "B"]	[('A', 0, 0), ('B', 4, 1), ('C', 2, 2), ('D', 3, 3), ('E', 1, 4)]	Elements were sorted according to the given list of **indices**. Numbers in black are their original indices, numbers in red are their new indices.
indices = [0.0, 4.0, 2.0, 3.0, 1.0] **items** = ["zero", "four", "two", "three", "one"]	AssertionError	The second example fails because elements in **indices** are floats
indices = [0, 4, 2, 3, 0] **items** = ["zero", "four", "two", "three", "one"]	AssertionError	The third example fails because **indices** doesn't contain numbers 0~4.

We recommend that you complete the function with for loop first, then convert the loop to list comprehension. Function `sort_by_index_with_for_loop` is provided on the starter code, but won't be graded.

Requirements: assert statements, one-line list comprehension.

HINT: You might want to use `range()` in your list comprehension.

```
def unusual_sort(indices, items):
    """
    >>> unusual_sort([0, 4, 2, 3, 1], \
        ["zero", "four", "two", "three", "one"])
    [('zero', 0, 0), ('one', 4, 1), ('two', 2, 2), \
('three', 3, 3), ('four', 1, 4)]

    >>> unusual_sort([0.0, 4.0, 2.0, 3.0, 1.0], \
        ["zero", "four", "two", "three", "one"])
    Traceback (most recent call last):
    ...
    AssertionError

    >>> unusual_sort([0, 4, 2, 3, 0], \
        ["zero", "four", "two", "three", "one"])
    Traceback (most recent call last):
    ...
    AssertionError

    >>> unusual_sort([0, 4, 2, 3], \
        ["zero", "four", "two", "three", "one"])
    Traceback (most recent call last):
    ...
    AssertionError
```

```
    """
```

Question 3:

Write a function `change_input` that takes in a list of strings and decode each string using the following steps in the following steps:

1. Each digit (0-9) multiplied by two
2. Lowercase vowels (a, e, i, o, u) are converted to UPPERCASE
3. Everything else (lowercase, spaces, punctuations, etc.) stay at the same position relative to each other.

Requirements: Assert statements, List Comprehension

Example:
Input: `["3.14IS PiE", "11My aGe iS"]`
Output: `['6.28IS PIE', '22My AGE IS']`

Explanations:
1. Numbers are marked blue
2. Lowercase vowels are marked green

Hints:
1. String methods in this list might be useful.

```
def change_input(strange_list):
    """
    >>> change_input(["3.14IS PIE", "11My aGe iS"])
    ['6.28IS PIE', '22My AGE IS']
    >>> change_input(["go t6o sleep at ", "5i like to start work
before "])
    ['gO t12O slEEp At ', '10I lIkE tO stArt wOrk bEfOrE ']
    >>> change_input("11My aGe iS")
    Traceback (most recent call last):
    ...
    AssertionError
    """
```

Question 4:

This question is very similar to the previous question but with one additional requirement. Now write a function `change_input_even_more` that takes in a list of strings and decode each string using the following steps in the following steps:

1. **Each number (0-9) is moved to the end of the string** *(this part is new)* and multiplied by two
2. Lowercase vowels (a, e, i, o, u) are converted to UPPERCASE
3. Everything else (lowercase, spaces, punctuations, etc.) stay at the same position relative to each other.

Requirements: Assert statements, List Comprehension

Example:
Input: `["3.14IS PIE", "11My aGe iS"]`
Output: `['.IS PIE628', 'My AGE IS22']`

Explanations:
1. Numbers are marked blue;
2. Lowercase vowels are marked green;
3.
4. Notice how we multiplied each individual number by 2, and how the point in 3.14 did not move.

Hints:
1. String methods in this list might be useful.
2. You might need more than one list comprehension for this problem.

```
def change_input_even_more(strange_list):
    """
    >>> change_input_even_more(["3.14IS PIE", "11My aGe iS"])
    ['.IS PIE628', 'My AGE IS22']
    >>> change_input_even_more(["go t6o sleep at ", "5i like to start
work before "])
    ['gO tO slEEp At 12', 'I lIkE tO stArt wOrk bEfOrE 10']
    >>> change_input_even_more("11My aGe iS")
    Traceback (most recent call last):
    ...
    AssertionError
    """
```

Question 5:

One of the team members decided to pick up lunch for everyone, but on his way he realized that his car was very low on gas. *Where should he stop to refuel?*

Question 5.1:

Write a function that takes:

- a non-negative integer representing the remaining mileage of his vehicle
- a dictionary, where the keys are gas brand names and the values are lists of 2-tuples representing the (distance, price) of each station under that brand.

The function should return the brand name with the **lowest** price among the **reachable** stations. If there is a tie, return any of them. Assume that there is at least one reachable station among all the provided options.

Requirement: **List Comprehension**

Note: **No** assert statements needed.

Example:

```
gas_stations = {
    'Shell': [(20, 5.2), (30, 5.3), (50, 5.6), (80, 5.3)],
    'Chevron': [(10, 5.8), (60, 5.7)],
     'Arco': [(20, 5.3), (10, 5.4)]
}
```

Input	Explanation
`mileage = 10`	The reachable stations are: `'Chevron': [(10, 5.8)]` (lowest 5.8) `'Arco': [(10, 5.4)]` (lowest 5.4) `'Arco'` has the lowest price
`mileage = 20`	The reachable stations are: `'Shell': [(20, 5.2)]` (lowest 5.2) `'Chevron': [(10, 5.8)]` (lowest 5.8) `'Arco': [(20, 5.3), (10, 5.4)]` (lowest 5.3) `'Shell'` has the lowest price

Hints:

1. You may find the built-in functions `min()` useful when finding the minimum price.
2. It may be helpful to create a **list of tuples** (this can be done using a list comprehension) to pair each price with its corresponding brand name.

   Note: When sorting or finding the minimum of a list of tuples, comparisons are made by the first elements of each tuple. If there is a tie, Python compares the second elements, and so on.

   Example: `min([(2, 'a'), (1, 'b'), (3, 'c')])` returns `(1, 'b')`.

3. Carefully plan your approach before writing any code. You should be able to implement this using no more than three list comprehensions (some of which may be nested). My solution uses fewer than three.

```
def cheapest_gas(gas_stations, mileage):
    """
    >>> gas_stations = { \
        'Shell': [(20, 5.2), (30, 5.3), (50, 5.6), (80, 5.3)], \
        'Chevron': [(10, 5.8), (60, 5.7)], \
        'Arco': [(20, 5.3), (10, 5.4)] \
    }
    >>> cheapest_gas(gas_stations, 10)
    'Arco'
    >>> cheapest_gas(gas_stations, 20)
    'Shell'
    """
```

Question 5.2 (Optional, Extra Credit):

Write a function that takes:

- a non-negative integer representing the remaining mileage of his vehicle
- a dictionary, where the keys are gas brand names and the values are lists of 2-tuples representing the (distance, price) of each station under that brand.

Returns the brand name with the **lowest average price among reachable stations.** In the event of a tie, return any of the tied options. Assume that at least one reachable station is provided.

Requirement: **List Comprehension only**

Notes:

1. No style requirements on this EC question only, no asserts needed either;
2. Since this is an extra credit question, tutors will not be able to help. Take Question 5.1 as a hint and build upon it!

Example:

```
gas_stations = {
    'Shell': [(20, 5.2), (30, 5.3), (50, 5.6), (80, 5.3)],
    'Chevron': [(10, 5.8), (60, 5.7)],
     'Arco': [(20, 5.1), (10, 5.4)]
}
```

Input	Explanation
`mileage = 10`	The reachable stations are: `'Chevron': [(10, 5.8)]` (average 5.8) `'Arco': [(10, 5.4)]` (average 5.4) `'Arco'` has the lowest average price
`mileage = 20`	The reachable stations are: `'Shell': [(20, 5.2)]` (average 5.2) `'Chevron': [(10, 5.8)]` (average 5.8) `'Arco': [(20, 5.1), (10, 5.4)]` (average 5.25) `'Shell'` has the lowest average price

```python
def cheapest_average_gas(gas_stations, mileage):
    """
    >>> gas_stations = { \
        'Shell': [(20, 5.2), (30, 5.3), (50, 5.6), (80, 5.3)], \
        'Chevron': [(10, 5.8), (60, 5.7)], \
        'Arco': [(20, 5.1), (10, 5.4)] \
    }
    >>> cheapest_average_gas(gas_stations, 10)
    'Arco'
    >>> cheapest_average_gas(gas_stations, 20)
    'Shell'
    """
```

Question 6

Your co-worker finally got gas but messaged you again! He needs help managing the orders because it's getting a bit chaotic. A new order needs to be created.

Write a function called `new_orders(orders, action, dish_name, amount)` to manage the lunch orders.

- `orders` is represented as a dictionary where the keys are food items (strings), and the values are quantities (positive integers).
- `action`: a specific operation (represented as a string).
- `dish_name`: the food item to update (represented as a string).
- `amount`: the number of that food item to update (represented as a non-negative integer).

The function should work as follows:

- If the action is "`add`", increase the quantity of the specified food item in the orders by the given quantity.
- If the action is "`remove`", decrease the quantity by the given amount. If the requested quantity to remove is greater than what's available, the order should be set to 0.

Assumptions:

- `dish_name` exists in `inventory`.
- There will only be two actions possible ("*add*" and "*remove*").

Your function should return a dictionary with the updated values. Construct the returned dictionary with a *one-line dictionary comprehension* (will be explained in the discussion), which is a dictionary version of list comprehension. The format is:

{key : value for ... in ...}

Requirements: Assert statements, Dictionary Comprehension

```
def new_orders(orders, action, dish_name, amount):
    """
    >>> orders = {'pizza': 10, 'burger': 5}
    >>> new_orders(orders, 'add', 'pizza', 5)
    {'pizza': 15, 'burger': 5}

    >>> new_orders(orders, 'remove', 'burger', 3)
    {'pizza': 10, 'burger': 2}

    >>> new_orders(orders, 'remove', 'pizza', 15)
```

```
{'pizza': 0, 'burger': 5}

>>> new_orders([], 'remove', 'burger', 3)
Traceback (most recent call last):
...
AssertionError
"""
```

Submission

Please submit the homework via Gradescope. You may submit more than once before the deadline, and only the final submission will be graded. Please refer to the Gradescope FAQ if you have any issues with your submission.